ADVANCED EXCEL · LESSON 14 OF 16

Excel Macros and VBA: Recording and Writing Automation

Automate a repeated monthly report. Record a macro, read the code it writes, change it to work on any data, add a loop over several sheets, and keep macro files safe from untrusted code.

Gap-fill lesson | Excel for Microsoft 365, 2026 | Advanced | 150 min

By the end of this lesson you can

- Record a macro and explain the code it creates
- Replace hard-coded cell addresses with ranges that adapt to the data
- Write a VBA loop that processes several sheets
- Save macro-enabled workbooks safely and recognise risky macros

Datasets used

`superstore_sales.csv`

What a macro is

A **macro** is a saved set of steps. Excel can record your actions and store them as code in the **VBA** language (Visual Basic for Applications). Running the macro repeats the steps exactly. VBA is the older but still widely used way to automate Excel on the desktop.

Use a macro when:

- the same steps happen every week or month,
- the steps are mechanical (format, filter, copy, save as PDF),
- the same person or team would otherwise repeat them by hand.

CASE STUDY **The monthly regional pack**

Every month, a finance analyst receives a sales sheet with a different number of rows. The steps are always the same: remove blank rows, format the header, add a total row, split the data into one sheet per region and save each as a PDF. It takes three hours by hand. A tested macro does it in under a minute and does it the same way every time.

Step 1: enable the Developer tab

The Developer tab is hidden by default. Go to **File > Options > Customize Ribbon** and tick **Developer**. The tab then shows **Record Macro**, **Visual Basic** and **Macros**.

Step 2: record a simple macro

1. Go to **Developer > Record Macro**.
2. Name it `FormatSalesTable`, store it in **This Workbook**, and give it a shortcut such as Ctrl+Shift+F.
3. Select a table, make the header bold with a fill colour, add a border, and set the number format on the sales column.
4. Click **Stop Recording**.

Now open the code with **Alt+F11** (the Visual Basic Editor) and find the module called `Module1`. It will look something like this:

VBA

```
Sub FormatSalesTable()
    Range("A1:F1").Select
    With Selection
        .Font.Bold = True
        .Interior.Color = RGB(30, 142, 90)
    End With
    Range("E2:E100").Select
    Selection.NumberFormat = "#,##0.00"
End Sub
```

The recorder wrote the exact addresses you clicked. That works once, on this data. It breaks when the data grows or moves.

::::warn Recorded macros are fragile A recorded macro uses cell addresses, `Select` and `Selection`. These depend on where the data was when you recorded. Read the code and rewrite it to use ranges and objects, as shown below. ::::

Step 3: make the macro adapt to the data

The same task, written to work on any table size, finds the last row and column itself:

```vba
Sub FormatSalesTable()
    Dim ws As Worksheet
    Dim lastRow As Long
    Dim lastCol As Long

    Set ws = ActiveSheet
    lastRow = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    lastCol = ws.Cells(1, ws.Columns.Count).End(xlToLeft).Column

    With ws.Range(ws.Cells(1, 1), ws.Cells(1, lastCol))
        .Font.Bold = True
        .Interior.Color = RGB(30, 142, 90)
        .Font.Color = RGB(255, 255, 255)
    End With

    ws.Range(ws.Cells(2, 1), ws.Cells(lastRow, lastCol)).Borders.LineStyle = xlContinuous
End Sub
```

What changed:

- `Set ws = ActiveSheet` stores the sheet in a variable, so later lines refer to it clearly.
- `End(xlUp)` moves up from the bottom of column 1 to the last filled row.
- No `Select` is needed. `Range` and `Cells` work on the object directly, which is faster and less error-prone.

Step 4: remove blank rows and add a total

Blank rows in exported data break sorts and totals. The macro below deletes any row where the first column is empty, working from the bottom up so that row numbers do not shift during the loop.

VBA

```
Sub RemoveBlankRows()
    Dim ws As Worksheet
    Dim r As Long

    Set ws = ActiveSheet
    For r = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row To 2 Step -1
        If Application.WorksheetFunction.CountA(ws.Rows(r)) = 0 Then
            ws.Rows(r).Delete
        End If
    Next r
End Sub
```

::::tip Loop backwards when deleting Deleting rows changes the row numbers below them. Looping from the bottom up (`Step -1`) avoids skipping rows. Looping from the top is a common bug that leaves some blanks in place. ::::

Step 5: loop over several sheets

The monthly pack needs the same formatting on every regional sheet. A `For Each` loop applies the macro to each sheet in the workbook.

VBA

```
Sub FormatAllSheets()
    Dim ws As Worksheet
    Application.ScreenUpdating = False
    For Each ws In ThisWorkbook.Worksheets
        If ws.Name <> "Instructions" Then
            ws.Activate
            Call FormatSalesTable
        End If
    Next ws
    Application.ScreenUpdating = True
    MsgBox "Formatting complete."
End Sub
```

`Application.ScreenUpdating = False` makes the macro much faster on large workbooks, because Excel stops redrawing the screen while it works. Always set it back to `True` at the end; the last line does this here.

Step 6: error handling

Macros fail in the middle of a run, which can leave a workbook half-changed. Add an error handler so that the user sees a clear message and screen updating is restored:

```vba
Sub SafeRun()
    On Error GoTo Fail
    Application.ScreenUpdating = False
    Call FormatAllSheets
    Application.ScreenUpdating = True
    Exit Sub
Fail:
    Application.ScreenUpdating = True
    MsgBox "The macro stopped: " & Err.Description, vbExclamation
End Sub
```

Before running any macro that changes data, save a copy. A macro cannot always be undone.

Saving and running safely

- Save a workbook that contains macros as **.xlsm** (Macro-Enabled Workbook). A plain .xlsx file cannot hold macros.
- Excel blocks macros by default. Enable them only for files you trust, from people you know.
- Macros can run any action your Windows account can, including deleting files. Treat a macro from an email attachment or a download with the same caution as an unknown program.
- Keep the code in the workbook you use, or in a shared personal macro workbook, and document what each macro does at the top of its code.

::::warn Risky macros A macro that calls `Shell`, `Kill`, or downloads files from the internet is doing something beyond formatting. Read the code line by line before you enable it, or ask someone who can. ::::

Macros versus other tools

Need	Best tool
One repeated desktop task	Macro (VBA)
Automation on Excel for the web	Office Scripts (next lesson)
Repeatable data cleaning and combining files	Power Query
Scheduled, cross-app workflows	Power Automate or n8n

::::key What to remember - Record a macro, then read and rewrite the code: replace `Select` with object references and fixed addresses with `End(xlUp)`. - Loop backwards when deleting rows. - Use `ScreenUpdating = False` for speed and always restore it, with an error handler. - Save macros in .xlsm files and enable them only for trusted sources. ::::

Quick quiz

5 questions · tap an answer to check it

1. Which key combination opens the Visual Basic Editor?

A Ctrl+E

B Ctrl+Shift+F

C Alt+F11

D F2

Answer: C. Alt+F11 opens the VBA editor.

2. Why is a recorded macro often fragile?

A It uses fixed addresses and Select, which break when the data changes size or position

B It cannot be saved

C It runs slower than a formula only

D It works only in Excel for the web

Answer: A. Recorded code refers to the exact cells selected.

3. When deleting rows in a loop, why loop from the bottom up?

A It is faster in every case

B Deleting shifts the row numbers below, so going up avoids skipping rows

C Excel requires it

D It sorts the data

Answer: B. Going up keeps the row numbers you have yet to process unchanged.

4. Which file extension is needed to save a workbook with macros?

A .xlsx

B .csv

C .xlsm

D .pdf

Answer: C. Only macro-enabled workbooks keep VBA code.

5. What does `Application.ScreenUpdating = False` do?

A It turns off the monitor

B It stops Excel redrawing the screen, so the macro runs faster

C It locks the workbook

D It deletes errors

Answer: B. Restore it to True at the end of the macro.

EXERCISE **A tidy-up macro**

Write a macro called `TidyPack` that, for every sheet except one called "Summary": (a) deletes rows that are completely blank; (b) makes the header row bold; (c) freezes the header row; and (d) shows a message box with the number of sheets it processed. Test it on a copy of a workbook with three sheets and some blank rows.

Show solution (worked answer)

Hints: loop over `ThisWorkbook.Worksheets` with `For Each`, reuse the backwards loop from Step 4, use `ws.Activate` then `ActiveWindow.FreezePanes` after setting `ActiveWindow.SplitRow = 1` and `ActiveWindow.FreezePanes = True`, and count the sheets in a variable that you display at the end.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.